

Roof Segmentation Methods and Results

We developed a roof segmentation pipeline from 30 satellite images and 25 supplied masks. After excluding one mismatched label, we used 24 labelled images for development and final training. The selected recipe uses a U-Net with an ImageNet-pretrained ResNet18 encoder, mild crop and resize augmentation, higher fine-tuning learning rates, and a prediction threshold of 0.45. The final model has completed 196 epochs on all 24 usable labels. Its downloaded export includes model checkpoints and predictions for the five unlabelled images. Cross-validation supports the development choices.

Project Roadmap

Stage	Outcome
Data preparation	Binary labels, validity masks, image review and exclusion of pair 278.
Validation design	Five frozen folds over 24 images; source and prepared files fingerprinted.
Model and checks	Model/loss checks, two-image memorization test and resume/inference checks.
Augmentation comparison	Mild cropping improved mean IoU in the reported comparison; retained.
Training diagnostics	Replaced IoU-based stopping with validation-loss monitoring and a minimum epoch floor.
Learning-rate comparison	3e-5 encoder / 3e-4 decoder improved loss and IoU in all five folds.
Threshold exploration	Broad high-IoU region; retained 0.45 instead of chasing a tiny peak.
Final fit and prediction	196 epochs, five prediction sets and a locally downloaded export.

How to Read the Results

Experiment comparisons below summarize the recorded development runs. The initial crop comparison and the later learning-rate comparison are separate experiments; their absolute scores should not be treated as a single controlled progression. Configuration and fold metadata remain the authority for reproducing each run.

Table of Contents

1	Data Preparation and Validation Folds.....	2
2	Model Implementation and Training Diagnostics.....	3
3	Augmentation and Learning Rate Experiments.....	4
4	Threshold Selection and Final Training Recipe	5
5	Execution Issues and Reproducibility	6
6	Final Deliverables and Evaluation Limits	7
7	Final Training Results	8
8	Conclusion.....	11

1 Data Preparation and Validation Folds

Label Quality and Valid Pixels

All supplied images and labels are 256×256 pixels. Images are RGBA and labels are greyscale. Approximately 4.2% of label pixels have intermediate intensities. We convert roof masks with a fixed threshold of 128, retain fully opaque pixels as valid, and exclude invalid pixels from the loss and metrics. Invalid RGB values are reset consistently before ImageNet normalization.

Pair 278 was quarantined: its mask is identical to the mask for 270 but does not align with image 278, including roof regions over a road. We preserve the original files and exclude the pair rather than shifting or fabricating a correction. This leaves 24 usable labels and five genuinely unlabelled images: 535, 537, 539, 551 and 553.

Visual and candidate-overlap review found no confirmed repeated scene among the supplied images. Group assignments reflect visual review, not verified geographic separation. Annotation consistency, particularly small roofs, occlusion and boundaries, remains a limitation; no comprehensive relabelling is claimed.

Five Fixed Folds

Fold	Validation IDs	Train / validation
0	121, 241, 328, 417	20 / 4
1	270, 301, 308, 314, 317	19 / 5
2	287, 320, 343, 345, 379	19 / 5
3	284, 300, 303, 315, 324	19 / 5
4	272, 274, 337, 381, 532	19 / 5

The seed-42 plan is saved in splits/validation_folds.json. Every eligible image is held out exactly once. Augmentation is generated only after splitting and only for training images. Checks reject changed eligibility, group assignments or file hashes. Every fold starts from fresh pretrained encoder weights and a newly initialized decoder and optimizer.

Augmentation

The baseline uses independent horizontal and vertical flips with probability 0.5, uniform quarter-turn rotations, and joint brightness/contrast adjustment with probability 0.8; both colour factors are sampled from 0.9–1.1.

The selected crop setting has probability 0.3. A square side of 224–256 pixels is sampled, then resized to 256×256 . RGB, target and validity share the same geometry. Crops require at least 70% valid pixels, with 10 attempts before reverting to the uncropped image. Background-only crops are allowed. RGB uses validity-weighted bilinear interpolation; masks use nearest-neighbour interpolation. Elastic/grid warps, erasing and noise remain disabled. Validation and inference never use random crops.

2 Model Implementation and Training Diagnostics

Network and Loss

The implemented network is a U-Net with a pretrained ResNet18 encoder and a compact GroupNorm decoder, producing one roof logit per pixel. Encoder BatchNorm statistics remain fixed during both warm-up and fine-tuning. Training uses AdamW, batch size 4 and weight decay $1e-4$. Five decoder-only warm-up epochs use learning rate $1e-3$ before unfreezing the encoder.

The loss is $0.5 \times$ masked binary cross-entropy plus $0.5 \times$ soft Dice loss, with smoothing $1e-6$. BCE consumes raw logits; soft Dice uses sigmoid probabilities. Both exclude invalid pixels; samples without valid pixels are excluded. Validation loss is computed over the full fold, avoiding dependence on the size of the last validation batch.

CPU Verification before Cloud Training

Local CPU tests checked loss masking, metrics, model forward/backward passes, frozen BatchNorm behaviour and checkpoint reloading. The two-image memorization test used training images 270 and 314 without augmentation. It passed at step 50, reaching mean IoU approximately 0.9648. This was a pipeline diagnostic, not held-out performance. Subsequent short CPU tests exercised interruption/resume, final fitting, prediction exports and threshold scoring.

Diagnosing Zero IoU

In the initial fold-4 run, validation IoU remained zero for 20 epochs at threshold 0.5. The last model also predicted no training roofs at that threshold, but mean probabilities were 0.4643 on training roof pixels versus 0.1474 on background, and 0.4429 versus 0.1550 on validation pixels. The network was learning separation that binary IoU could not yet reveal.

The original stopping rule monitored IoU and retained epoch 1 when every score tied at zero. We changed checkpoint selection and stopping to validation BCE + soft Dice loss, added `min_epochs=30` and `patience=20`, and logged predicted roof fraction. Patience counts non-improving epochs after warm-up, including those before the minimum-epoch floor. The implementation has no minimum-improvement tolerance, so very small loss improvements can reset patience.

After 100 epochs, one run still predicted no roofs at 0.5, despite a best validation loss of 0.3956 at epoch 98. Its saved masks reached fold-4 mean IoU 0.6866 at 0.45. Longer training alone had not solved the probability-cutoff issue. This motivated separate evaluation of learning rates and prediction thresholds.

Training Loss and Prediction Threshold

The threshold converts probabilities into binary masks; it does not enter BCE or soft Dice. With `monitor="val_loss"`, changing the threshold affects reported IoU, Dice, precision, recall and masks, but not gradients, early stopping or checkpoint selection. With `monitor="val_iou"`, it would indirectly affect checkpoint selection. Our selected procedure uses validation loss.

3 Augmentation and Learning Rate Experiments

Mild Cropping Compared with the Baseline

In the initial paired comparison, mean per-image IoU increased from 0.7305 to 0.7511, a gain of 2.06 percentage points. Fifteen of 24 images improved and four of five folds improved. Fold 4 declined by 1.66 points. The largest gain was image 328; the largest declines were 274 and 532. Image 270 remained difficult for both methods.

Side-by-side image, label, prediction and error-map review supported keeping cropping. The additional-seed check was deliberately skipped. Consequently, the crop effect is promising evidence from one seed, not a demonstrated seed-independent benefit.

Paired Learning Rate Comparison with Cropping Enabled

Both runs used the same frozen folds, crop probability 0.3, threshold 0.45, seed, warm-up and validation-loss stopping rules, with a 500-epoch ceiling. The reference fine-tuning rates were encoder 1e-5 / decoder 1e-4. The candidate multiplied both by three: 3e-5 / 3e-4. Decoder warm-up remained 1e-3. This experiment does not isolate the contribution of each individual learning rate.

Fold	Ref loss	3x loss	Ref IoU	3x IoU	Best epochs ref / 3x
0	.1639	.1192	.8275	.8424	341 / 173
1	.2482	.2053	.7300	.7505	217 / 138
2	.1736	.1405	.7898	.8078	424 / 207
3	.2838	.1646	.7043	.7728	186 / 196
4	.3042	.2329	.7960	.8250	329 / 304

Mean validation loss across folds fell from 0.2347 to 0.1725, a reduction of 0.0622. Mean per-image IoU increased from 0.7671 to 0.7979, a gain of 3.08 percentage points. Every fold improved in both measures; 19 of 24 images improved in IoU. The candidate reached its best checkpoint earlier in four folds.

Loss values above are equally weighted fold summaries. Overall IoU averages the 24 images equally, rather than weighting the four-image and five-image folds equally. All figures describe selected development checkpoints, not the final all-label model.

Selected Learning Rates

Retain the higher fine-tuning rates as the best of the two settings tested. Stop the learning-rate search and inspect thresholds using the candidate's saved out-of-fold predictions. Additional architecture, distortion, crop-strength and warm-up searches were discussed but were not performed.

4 Threshold Selection and Final Training Recipe

Threshold Exploration

We swept thresholds from 0.20 to 0.70 in steps of 0.025 on the selected higher-rate crop model. No retraining was required: each fold's saved probabilities were converted to masks at each cutoff. Mean per-image IoU was the threshold-selection score, with Dice, precision, recall and per-fold curves as diagnostics.

Threshold	Mean IoU	Dice	Precision	Recall
.200	.7930	.8820	.8815	.8919
.450	.7979	.8847	.9021	.8768
.500	.7981	.8847	.9050	.8743
.575	.7984	.8848	.9096	.8701
.600	.7981	.8845	.9111	.8681
.625	.6248	.6957	.7227	.6771

The grid peak was 0.575, but its IoU gain over 0.45 was only about 0.0005, or 0.05 percentage points. We retained 0.45 within the broad stable region, favouring slightly higher recall and avoiding a change for a negligible gain. The abrupt decline above 0.60 warrants per-fold inspection; its exact image-level cause is not established by the aggregate table.

Empty-mask metric conventions matter: both masks empty gives IoU/Dice/precision/recall of 1; when labelled roofs exist but none are predicted, undefined precision is recorded as 0. Thus macro precision can fall abruptly if some images lose all predicted roofs. Images with no valid pixels are excluded.

Final Recipe

Setting	Selected value
Model and optimizer	Pretrained ResNet18 U-Net; AdamW; batch 4; weight decay 1e-4
Augmentation	Baseline flips, rotations and colour changes plus crop probability 0.3
Warm-up	5 epochs; frozen encoder; decoder LR 1e-3
Fine-tuning	Encoder LR 3e-5; decoder LR 3e-4; encoder BatchNorm fixed
Loss and output cutoff	Masked BCE + soft Dice, equal weights; threshold 0.45
Final fit	Seed 42; all 24 usable labels; fixed 196 epochs

The candidate's best epochs were 173, 138, 207, 196 and 304; the median is 196. This is a practical duration heuristic, not a proven optimum, since a final epoch includes more training images than a CV epoch. With all labels used for fitting, monitor, min_epochs and patience are inactive. The final checkpoint is the last epoch, not a validation-selected "best" checkpoint.

5 Execution Issues and Reproducibility

Local Preparation and Remote GPU Execution

The repository separates reusable code in `roof_utils/` from notebooks. A `.env` file defines raw data, prepared data, models and reports paths. Local CPU checks precede Colab GPU training through the VS Code notebook kernel. The remote runtime must have `pyproject.toml`, `roof_utils/`, `splits/`, `data/` and `data_preparation_outputs/`; selecting a kernel alone does not copy local files.

Files Uploaded but Modules Not Found

A separately uploaded `final_training.py` or `hyperparameter_review.py` could land in `/content` rather than the imported `roof_utils/` directory. The repair was to inspect the imported package path, place the module in that directory, and invalidate import caches or restart the kernel. Old `CVConfig` imports also caused an unexpected monitor argument error. Restarting and rerunning setup, with the current module in the correct location, resolves stale imports.

Interruptions and Incompatible Resumes

Each epoch writes `last.tmp` before replacing `last.pt`. An interrupt during writing can leave a partial temporary file while the preceding completed checkpoint remains available. Rerunning a matching experiment resumes from the last saved epoch; that interrupted epoch may repeat. Completed folds are reused. Only one process should write to a given run directory.

The runner records configuration, data, source and software signatures. Changes produce an explicit mismatch instead of mixing incompatible results. Use a new run name for a changed experiment; do not delete `experiment.json` alone. Adding or changing a utility module can alter the signature. Complete saved predictions remain usable for separate threshold analysis without retraining.

Reference Availability and Export

The threshold notebook can reuse a complete crop reference or train/resume one when absent. Missing prediction exports in a supposedly completed fold require restoring those files or choosing a new run name. Threshold sweeps and pair comparisons write separate reports, preserving original metrics.

Files under `/content` are normally temporary runtime storage. A ZIP created there is not yet a local backup or a Google Drive file. We export checkpoints, reports, utility code and a path template, then download the archive using the VS Code Colab Contents view. The raw/prepared dataset is retained separately. Local CPU tests used PyTorch 2.8.0; the final exported GPU run records PyTorch 2.11.0+cu128, so software versions must be retained with results.

6 Final Deliverables and Evaluation Limits

Verified Final Export

The run `final_crop_lr3x_t045_196epochs` completed 196 epochs on CUDA with the selected higher learning rates, crop probability 0.3 and threshold 0.45. The local export contains `final.pt` for inference, `last.pt` for resume, `experiment.json`, a 196-epoch history, `result.json` and all five prediction sets. The prediction metadata's checkpoint hash matches the downloaded `final.pt`.

The earlier `final_crop_t045_285epochs` export is retained; the 196-epoch higher-rate run is the selected model.

Image ID	Predicted roof fraction of valid pixels
535	20.29%
537	14.96%
539	12.50%
551	9.49%
553	13.65%

These fractions describe predictions, not accuracy. Outputs include probabilities, binary roof PNGs (255/0) and validity PNGs. Invalid pixels are zeroed; overlays support visual inspection.

Limits on the Conclusions

Only 24 labelled images underpin development. The same folds informed checkpoint, augmentation, learning-rate and threshold choices, so the selected scores can be optimistic. The five models have overlapping training sets; their results are not independent experimental repetitions. Geographic independence is unverified. One seed was used for the comparisons. There is no labelled test set for the five new predictions, and the final model has no held-out evaluation.

Method References

Method background: Ronneberger et al., U-Net (2015), arxiv.org/abs/1505.04597; PyTorch BCEWithLogitsLoss documentation; scikit-learn cross-validation guide.

7 Final Training Results

The selected final model was fitted to all 24 usable labelled images for 196 epochs, using crop probability 0.3, encoder learning rate $3e-5$, decoder learning rate $3e-4$ and prediction threshold 0.45. The figures in this section are the saved outputs of `final_model_training.ipynb` for `final_crop_lr3x_t045_196epochs`.

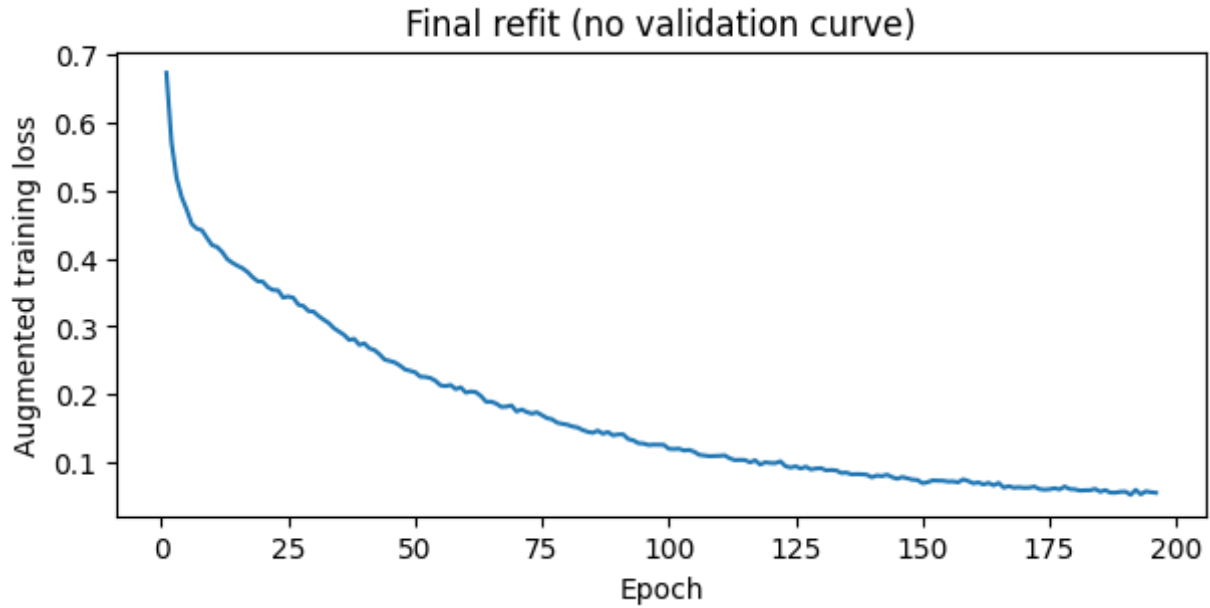


Figure 1. Augmented training loss during the final fit. No validation curve is available because all usable labels were included in training.

The recorded training loss decreases from 0.6723 in epoch 1 to 0.0559 in epoch 196. The curve shows successful optimization on augmented training examples, with progressively smaller changes later in training. It does not establish generalization or rule out overfitting. [The stopping point was fixed from the median best epoch of the crop model’s five higher-learning-rate validation runs, rather than selected from this training curve.](#)

How to Read the Prediction Panels

Each panel shows the input RGB image, the predicted roof probability map and the binary roof prediction overlaid in green. Purple indicates low roof probability and yellow indicates high probability on a common 0–1 scale. White areas in probability maps are masked invalid pixels; the corresponding missing regions appear black in the RGB images. Roof masks use the same global cutoff of 0.45 for every image.

These are predictions for previously unlabelled images, not comparisons against known masks. We therefore describe visible spatial agreement and remaining boundary questions qualitatively. No IoU, Dice or accuracy score is assigned to these five images.

Roof Predictions for Images 535 and 537

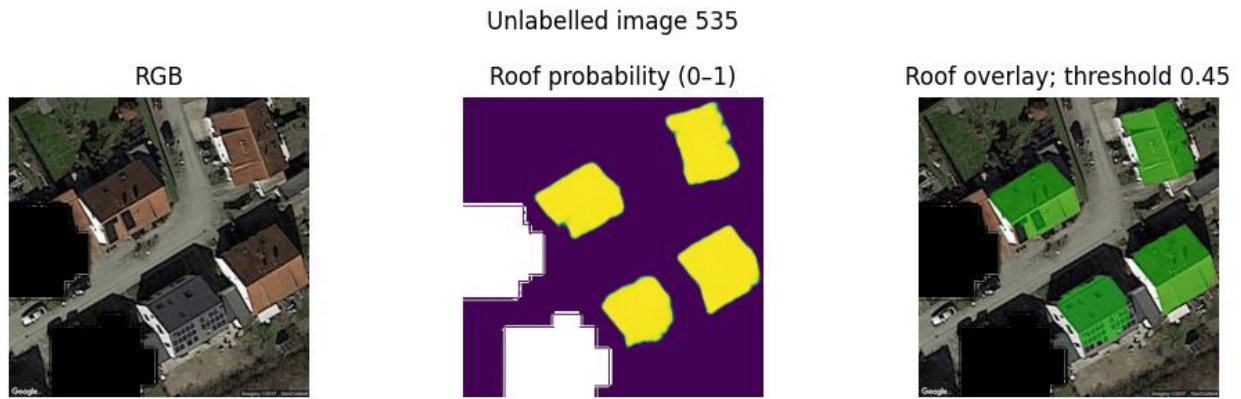


Figure 2. Image 535. The overlay identifies the four prominent visible roofs. Predicted roof pixels cover 20.29% of the valid region.

The probability map separates the main roof regions from the central road and nearby vegetation. The broad roof extents appear plausible, while detailed edges and small adjoining structures still require annotation-based checking.

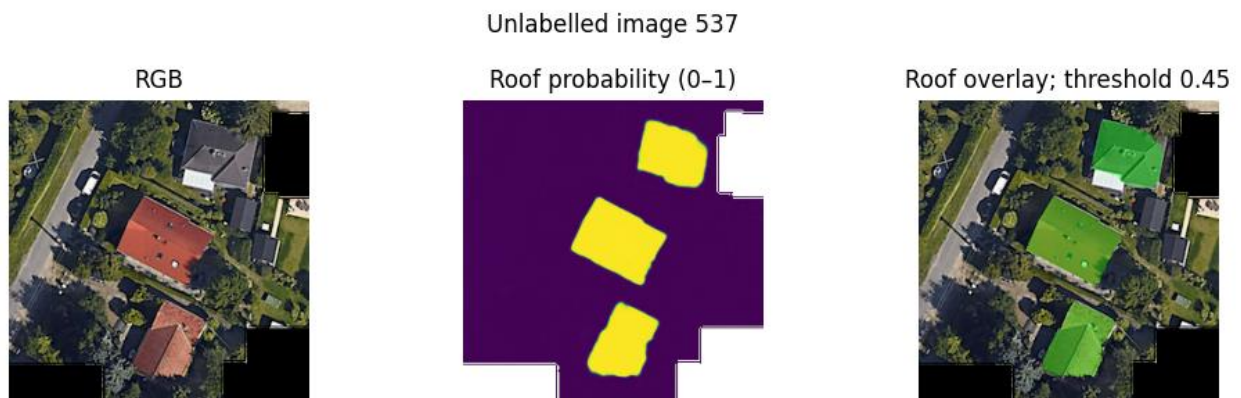


Figure 3. Image 537. The three main visible roofs receive distinct predicted regions. Predicted roof pixels cover 14.96% of the valid region.

The model identifies roofs with different colours and orientations. Fine boundary placement near vegetation and shadow remains uncertain. The mask gives a segmentation proposal, not evidence that every visible roof pixel is correctly classified.

Roof Predictions for Images 539 and 551

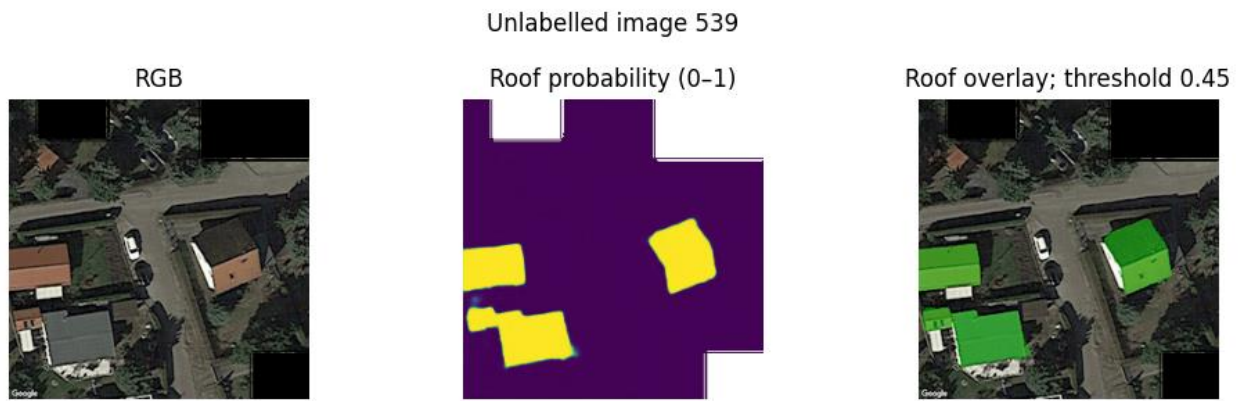


Figure 4. Image 539. Predictions include roof regions near the left image boundary and the separate roof on the right. Predicted roof pixels cover 12.50% of the valid region.

The road is largely separated from the predicted roofs. Structures cut by the image edge and narrow adjoining regions are useful targets for manual review because their full extent is not visible.

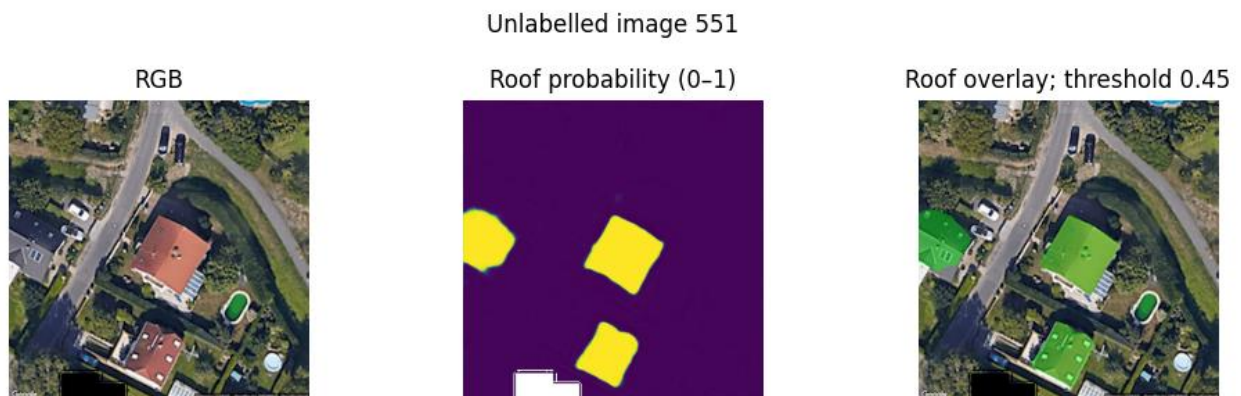


Figure 5. Image 551. The central and lower roofs, together with a partial roof at the left boundary, receive positive predictions. Predicted roof pixels cover 9.49% of the valid region.

The central roof remains distinct from the adjacent road and garden. Roof details, small extensions and boundaries next to trees need closer inspection before the masks could be treated as reference annotations.

Roof Prediction for Image 553

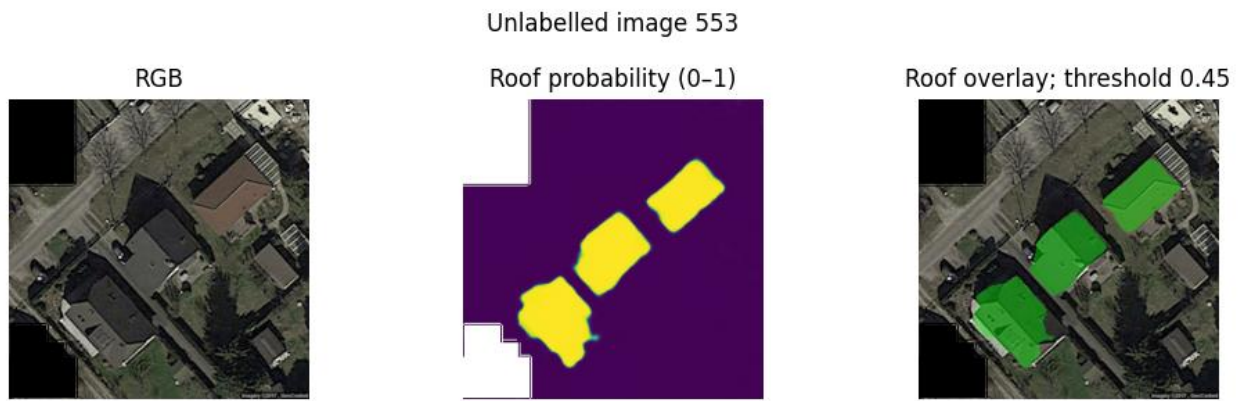


Figure 6. Image 553. The model predicts three prominent roof regions across the diagonal scene. Predicted roof pixels cover 13.65% of the valid region.

The overlay follows the broad extents of the three main roofs despite differences in appearance and orientation. The irregular lower roof boundary and nearby shadows illustrate where visual plausibility should be followed by independent label review.

8 Conclusion

We produced a complete, reproducible roof segmentation workflow: data and label checks, fixed validation folds, paired augmentation, transfer learning, model selection, final fitting and prediction export. Excluding the mismatched mask for image 278 avoided training on an unsupported target. Using validity masks throughout ensured that missing image regions did not contribute to the objective or evaluation.

Development experiments supported mild crop and resize augmentation and the higher encoder/decoder learning rates. In the matched learning-rate comparison, mean per-image IoU increased from 0.7671 to 0.7979 at threshold 0.45, and validation loss improved in every fold. The threshold sweep showed a broad stable region, so we retained 0.45 rather than selecting a slightly higher cutoff for a negligible score gain.

The final 196-epoch model generated coherent roof regions for all five unlabelled images and was exported with its configuration, weights, history, probabilities and masks. The training and prediction figures demonstrate that the implemented pipeline runs end to end and produces usable segmentation proposals. They do not provide an independent performance estimate for the final model.

The main limitations are the small labelled dataset, one experimental seed, unverified geographic independence and repeated model selection on the same development folds. Within the scope of the project, the selected model and documented exports provide a practical final result, with those limitations stated explicitly.